DC MOTOR SELF-TUNING PI CONTROL USING RECURSIVE LEAST SQUARES

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....6
Recommended Citation.....7

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Run a self-tuning PI speed controller, which combines recursive least squares (RLS) estimation with pole-placement design, in simulation and on the ACE-Lab hardware.	☐
2	Apply a change in the effective gain of the motor during operation, and explain how the estimated model and the controller gains respond.	☐
3	Investigate how the forgetting factor and the design target affect re-tuning, and evaluate the self-tuning controller against the engineering requirements.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercises, DC Motor Speed Control Using a PI Controller, Online Parameter Estimation of a DC Motor using Recursive Least Squares, and DC Motor PI Control Design by Pole Placement from an LLS Model. The set-up is not repeated here.

The model runs on an Arduino Mega 2560 at 100 Hz in external mode (Monitor & Tune), using the same pins as the ACE-Lab (Base + Sense) (5V, GND, D2, D3 and D5). The motor is driven through the Cytron Maker Drive from a 9V battery, and no more than 6V is applied to the motor (a PWM command of 0 to 170). Keep the folder structure of the supplied files: `hw_collect.m` uses `analyse_hw_run.m` from the `RL_PI_Tuner` folder.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. The PI controller in the practical exercise, DC Motor PI Control Design by Pole Placement from an LLS Model, was designed once, from a model identified before the

controller was used. If the motor changes, the gains are no longer correct. Here, the self-tuning PI controller described in the Adaptive Control: the Self-Tuning PI Controller Learning Guide re-identifies the motor every 0.05 s using RLS, and recalculates its gains by pole placement from the latest model.

The Self-Tuning PI Model

Open `dc_motor_str.slx`. Its structure is shown in Figure 1. The PI Controller block runs every 0.01 s using the velocity form, so that its output does not jump when the gains change. The RLS Estimator block estimates the model $y_k = -a_1 y_{k-1} + b_0 u_{k-1} + c$ every 0.05 s from the controller output, passed through the same filter as the speed, and the measured speed. At every sample, the Self-Tuning PI block converts the latest estimate into K and p and recalculates the gains by pole placement, provided that the model is valid ($0 < p < 1$ and $K > 0$) and within the gain limits.

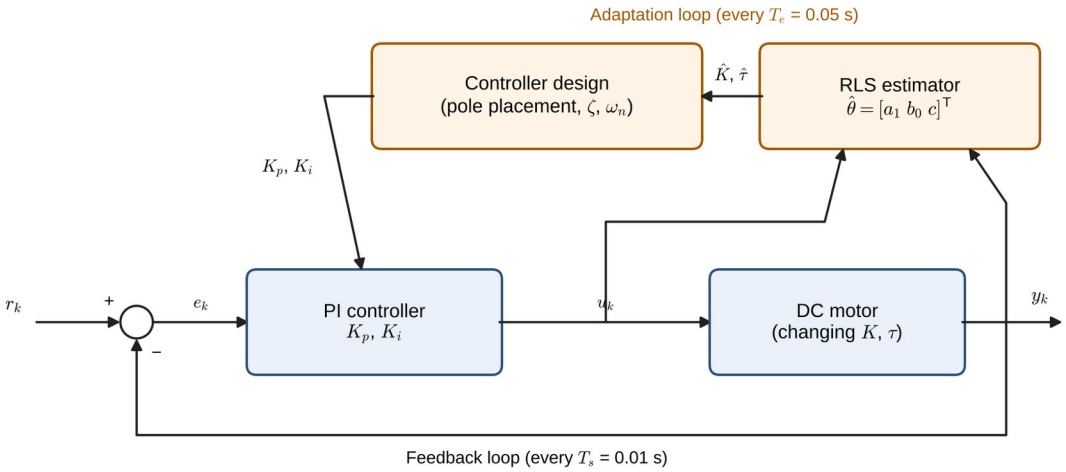


Figure 1: *Structure of the Self-Tuning PI Controller in dc_motor_str.slx*

To test re-tuning without changing the hardware, the model contains a software load, shown in Figure 2. From t = 40 s, the controller output is multiplied by a factor, α, before it is limited to the 0 to 6 V range of the motor. Since the motor sees α times the voltage the controller intended, this has the same effect on the loop as a change in the gain of the motor: α = 0.6 (scenario A) represents a load being added, and α = 2.0 (scenario B) a load being removed. The controller output limit is set to 10 V, so that the motor can still reach 6 V when α = 0.6.

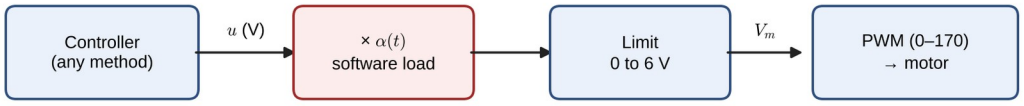


Figure 2: *The Software Load: the Controller Output is Scaled by α(t) Before the 0–6 V Motor Limit*

The parameters are stored in the model workspace (Modeling > Model Explorer > Model Workspace), see Table 2.

Table 2: *Main Parameters of dc_motor_str.slx (Hardware Model)*

Parameter	Value	Meaning
`T_exp`	100 s	Length of the run
`r_low`, `r_high`, `r_period`	70, 90 RPM, 6 s	Speed reference: a new step every 3 s
`zeta`, `wn`	1, 6 rad/s	Design target for pole placement
`adapt_on`	1	1 = self-tuning, 0 = fixed gains `Kp0`, `Ki0`
`Kp0`, `Ki0`	0.1, 0.5	Gains used until the model is valid
`Kp_max`, `Ki_max`	1, 10	Gain limits
`lambda`, `D`	0.99, 5	Forgetting factor (memory about 5 s), estimate every 0.05 s
`P0`, `TraceMax`	100, 10^4	Initial covariance, wind-up limit
`K0`, `tau0`	10 RPM/V, 0.3 s	Initial guess of the model
`t_change`, `K_factor`	40 s, 0.6	Software load: time and factor α
`Vs`, `V_motor`	10 V, 6 V	Controller output limit, motor voltage limit

Run the Self-Tuning Controller in Simulation

Before using the hardware, open `dc_motor_str_virtual.slx`, which replaces the Arduino blocks with a virtual motor (K = 16 RPM/V, τ = 0.25 s, 0.5 V dead-band) whose gain falls to 60% at t = 40 s, with a reference between 20 and 40 RPM.

1. Run the model (80 s), then run `plot_str_results.m` to plot the speed, voltage, PI gains and estimated model.
2. Record the estimated gain and the PI gains just before the change (at 39 s) and at the end of the run, and the RMS speed error between 20 and 40 s and between 60 and 80 s, in Table 3.

   Example results are shown in Figure 3. Before the change, the estimated gain is about 16 RPM/V and the controller uses $K_p \approx 0.12$ and $K_i \approx 0.54$. After the change, the estimate falls to 9.6 RPM/V, 60% of its original value, within about 12 s, and the gains rise to K_p = 0.20 and K_i = 0.92. The RMS speed error was 3.48 RPM before the change and 3.63 RPM after it, so the controller restored almost the same tracking performance.

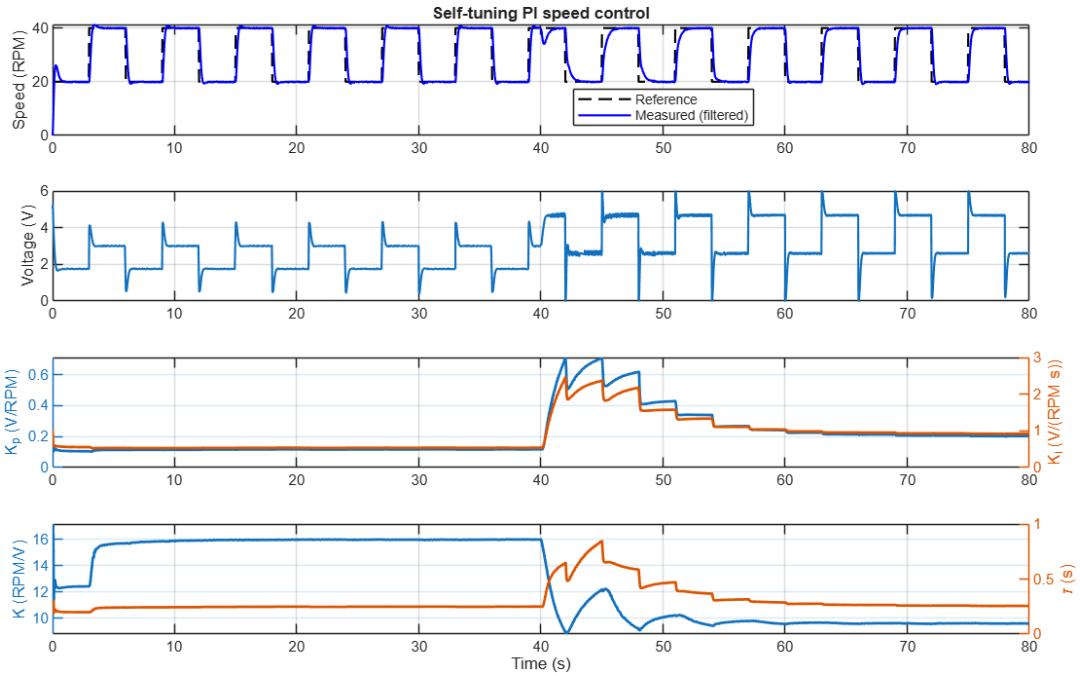


Figure 3: *Example Results from dc_motor_str_virtual.slx: Speed, Voltage, PI Gains and Estimated Model; the Motor Gain Falls to 60% at t = 40 s*

Table 3: *Self-Tuning PI Control of the Virtual Motor*

	$\hat{K}$ (RPM/V)	K_p (V/RPM)	K_i (V/(RPM s))	RMS error, 20–40 s (RPM)	RMS error, 60–80 s (RPM)
Example: at 39 s	16.0	0.12	0.54	3.48	–
Example: at 80 s	9.6	0.20	0.92	–	3.63
Your results: at 39 s					–
Your results: at 80 s				–	

Run the Self-Tuning Controller on the Hardware

1. Set up the model for scenario A and open it, using the supplied function `hw_setup`:

MATLAB Script: running the self-tuning PI controller on the hardware

```
hw_setup('RLS self-tuning PI', 'A');   % scenario A: alpha = 0.6 at 40 s
% ... press Monitor & Tune and wait for the 100 s run to finish ...
res = hw_collect();                    % save, check and plot the run

% Estimated motor gain before and after the load change
th = double(evalin('base', 'theta_est'));
if ndims(th) == 3, th = reshape(th, 3, [])';
elseif size(th, 2) ~= 3, th = th'; end
t  = (0:size(th, 1) - 1)'*0.01;
K  = th(:,2)./(1 + th(:,1));
Kb = mean(K(t > 30 & t < 40));  Ka = mean(K(t > 80));
Kib = mean(res.Ki(res.t > 30 & res.t < 40));  Kia = mean(res.Ki(res.t > 80));
fprintf('K: %.1f -> %.1f RPM/V (x%.2f); Ki: %.2f -> %.2f (x%.2f)\n', ...
        Kb, Ka, Ka/Kb, Kib, Kia, Kia/Kib);
```

2. Press **Monitor & Tune** when prompted, wait for the run to finish, and then run the rest of the script. `hw_collect` prints the worst-case performance before and after the load change, with PASS or FAIL for each requirement (rise time below 1 s,

steady-state error at most 0.1 RPM, overshoot at most 5%, and chatter at most 5 PWM counts per sample).

3. Repeat for scenario B, `hw_setup('RLS self-tuning PI', 'B')`, in which α = 2.0.
4. Record the factor by which the estimated gain and the integral gain changed in Table 4. Compare them with the true factor, α.

Example hardware results for scenario A are shown in Figure 4. When the software load is applied at 40 s, the speed falls below the reference and the controller output rises. As the estimate settles, the gains first rise sharply and then fall back over about 15 s, which causes a short period of chatter in the controller output. On an ACE-Lab motor, the estimated gain changed by a factor of 0.60 in scenario A and 1.99 in scenario B, against true factors of 0.6 and 2.0, and the integral gain moved inversely, by factors of 1.88 and 0.70.

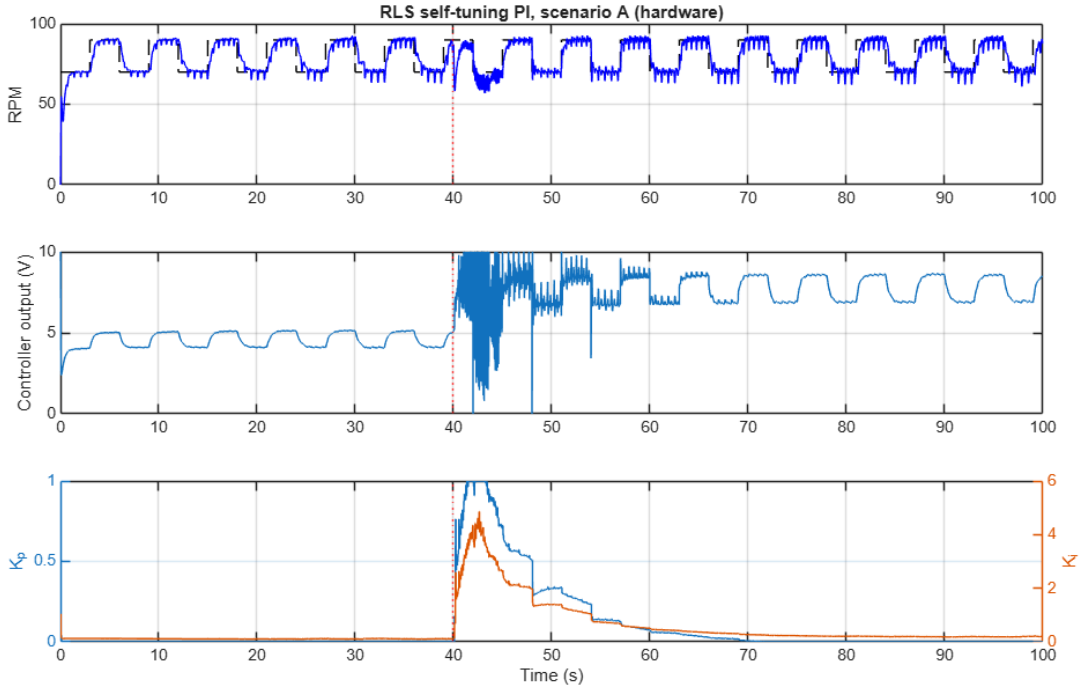


Figure 4: *Example Hardware Results for the Self-Tuning PI Controller in Scenario A: Speed (Top), Controller Output (Middle) and Gains (Bottom); the Software Load is Applied at t = 40 s*

Table 4: *Re-Tuning on the Hardware*

Scenario	True factor, α	Change in $\hat{K}$ (example)	Change in K_i (example)	Change in $\hat{K}$ (yours)	Change in K_i (yours)	R1–R4 met after the change?
A	0.6	×0.60	×1.88			
B	2.0	×1.99	×0.70			

In the example, the RMS speed error of the self-tuning controller decreased after both changes (by 10% in scenario A and 22% in scenario B). It met R1 and R4 after the change, but not R2 or R3. Use the questions at the end of this practical exercise to explain why.

Investigate the Forgetting Factor and the Design Target

1. Repeat scenario A with a longer memory: after `hw_setup`, set `lambda` to 0.995 (about 10 s) in the model workspace, using `assignin(get_param('dc_motor_str', 'ModelWorkspace'), 'lambda', 0.995)`. Then repeat with λ = 0.98 (about 2.5 s).
2. Return to λ = 0.99 and repeat scenario A with a faster design target, ω_n = 30 rad/s (`wn`). For this motor, ω_n = 6 rad/s gives $2\zeta\omega_n\tau < 1$, so the self-tuning controller also has $K_p = 0$ for most of the run.
3. Record your results in Table 5.

Table 5: *Effect of the Forgetting Factor and Design Target (Scenario A)*

λ	ω_n (rad/s)	Time to re-tune (s)	Change in $\hat{K}$	Worst rise time after (s)	Worst overshoot after (%)	Worst chatter after (counts)
0.99	6					
0.995	6					
0.98	6					
0.99	30					

Consider the following questions:

- How closely did the estimated motor gain follow the true load factor, and how long did it take?
- One encoder count per 0.01 s sample is about 1.68 RPM. What does this mean for the 0.1 RPM limit in R2, for this or any other controller?
- How much faster was the self-tuning controller with ω_n = 30 rad/s, and what did it cost in chatter?
- What would happen to the self-tuning controller if the reference stayed constant for a long time, and the load then changed?

4.0 Concluding Remarks

This practical exercise implemented a self-tuning PI speed controller on the ACE-Lab. Using MATLAB, Simulink and ACE-Lab hardware, a recursive least squares estimator identified the motor every 0.05 s, and pole placement recalculated the PI gains from the latest model. When the effective gain of the motor was changed during operation, the estimate followed the change to within about 1%, and the integral gain moved in the opposite direction to compensate, without any manual re-tuning.

This practical exercise also showed that adaptation does not remove the need for good design choices. The forgetting factor sets how quickly the controller re-tunes and how steady its gains are, and the design target still determines how fast the closed loop can be.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). DC Motor Self-Tuning PI Control using Recursive Least Squares. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671554